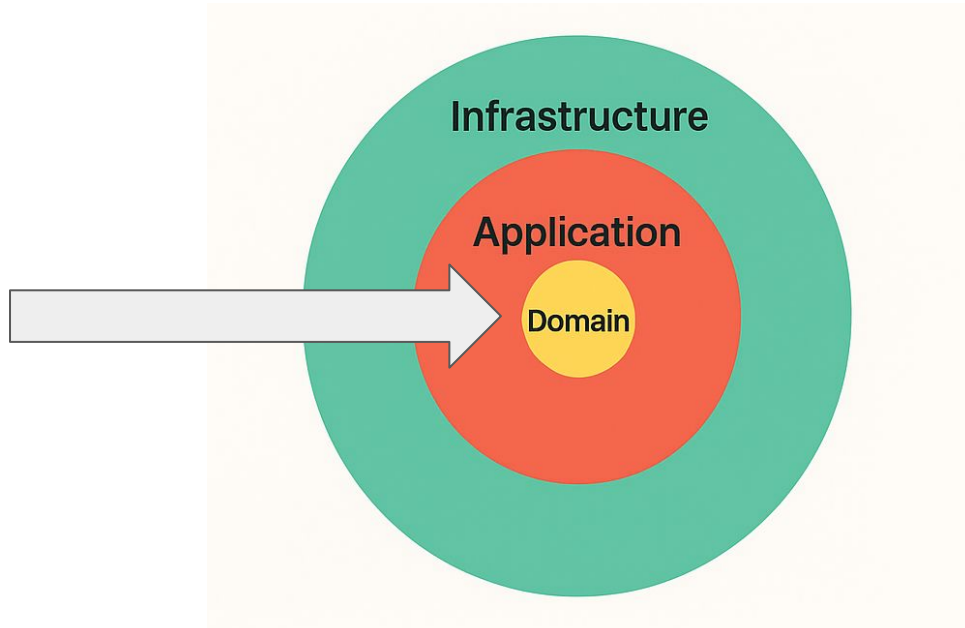


MÓDULO 2: ARQUITECTURA DE SOFTWARE

DOMINIO: ENTIDADES Y VALUE OBJECTS

APLICANDO CLEAN ARCHITECTURE CON TYPESCRIPT

Dominio: Entidades y Value Objects



Value Objects

- Inmutabilidad
- `create()`
- `equals()`
- invariants

Representación de un Value Object

```
// src/domain/value-objects/Currency.ts
export type Currency = "EUR" | "USD"

// src/domain/value-objects/Price.ts
import { Currency } from "../Currency"

export class Price {
  private constructor(readonly amount: number, readonly currency:
Currency) {}
  static create(amount: number, currency: Currency) {
    if (!Number.isFinite(amount) || amount < 0) throw new
InvalidPrice("Invalid amount")
    const rounded = Math.round(amount * 100) / 100
    return new Price(rounded, currency)
  }
  add(other: Price) {
    if (this.currency !== other.currency) throw new
CurrencyMismatch()
    return Price.create(this.amount + other.amount, this.currency)
  }
  multiply(qty: number) {
    if (!Number.isInteger(qty) || qty <= 0) throw new
InvalidQuantity()
    return Price.create(this.amount * qty, this.currency)
  }
  equals(other: Price) {
    return this.amount === other.amount && this.currency ===
other.currency
  }
}
```

Entidades/Aggregates

- Identidad
- Reglas internas
- Consistencia

Testing del Dominio

- Rápido
- Puro
- Significativo

Testing del Dominio

```
// tests/domain/price.spec.ts
import { describe, it, expect } from "vitest"
import { Price } from "../../src/domain/value-objects/Price"

describe("Price", () => {
  it("no permite negativos y redondea a 2 decimales", () => {
    expect(() => Price.create(-1, "EUR")).toThrow()
    const p = Price.create(12.345, "EUR")
    expect(p.amount).toBe(12.35)
  })
})

// tests/domain/order.spec.ts
import { Order, OrderId, CustomerId } from
"../../src/domain/entities/Order"
import { Price } from "../../src/domain/value-objects/Price"
import { Quantity } from "../../src/domain/value-objects/Quantity"
import { SKU } from "../../src/domain/value-objects/SKU"
```

Testing del Dominio

```
it("acumula total con misma moneda y emite eventos", () => {  
  const o = Order.create(OrderId("o-1"), CustomerId("c-1"))  
  o.addItem(SKU.create("abc-1"), Price.create(10, "EUR"),  
    Quantity.create(2))  
  o.addItem(SKU.create("abc-2"), Price.create(5, "EUR"),  
    Quantity.create(1))  
  expect(o.total().amount).toBe(25)  
  const ev = o.pullDomainEvents()  
  expect(ev.some(e => e.type === "order.created")).toBe(true)  
  expect(ev.some(e => e.type === "order.item_added")).toBe(true)  
})
```

Antipatrones

Primitive obsession: usar `string/number` en vez de VOs.

Setters mutables: exponen estados inválidos.

Poner lógica de negocio en **controllers** o **repositorios**.

Dominar con **enums** “**Dios**” y condicionales en cascada.

Igualdad de entidad por **valor** en vez de por **ID**.



IA como Copiloto

Descubrir invariantes

“Actúa como domain modeler. Dado este contexto {texto}, enumera invariantes candidatas y casos límite que deberían vivir en Value Objects. Propón nombres.”

Refactor a VOs (primitive obsession)

“Partiendo de este código con tipos primitivos, sugiere Value Objects inmutables con `create()` y `equals()`, más tests de construcción y operaciones.”

Eventos de dominio

“Para el agregado `Order`, propone eventos de dominio (nombre, payload mínimo) que describan cambios significativos. Asegúrate de que los eventos no contengan detalles de infraestructura.”

Revisión de errores

“Revisa esta jerarquía de errores de dominio y su uso. ¿Dónde usar excepciones y dónde `Result`? Propón una tabla de decisiones breve.”